



HTTPS, SSL/TLS & Secure Communication

Lecture 24

Mahhek Tahir

Why HTTP is insecure

How HTTPS protects data

Basic Nginx configuration



HTTP vs HTTPS

HTTP (Not Secure)

- Data sent in **plain text**
- Anyone can read it

HTTPS (Secure)

- Data is **encrypted**
- Safe communication

HTTP is like sending a postcard...

HTTPS is like sending a sealed envelope.



Problem with HTTP

User sends login:

username=ali&password=123456

Hacker can intercept

Password is visible

Without HTTPS, everything can be seen.



What is SSL/TLS?

Protocol used for **encryption**

SSL (old) → TLS (modern)

Used in HTTPS

SSL/TLS encrypts data between browser and server.

SSL → *Secure Sockets Layer*

TLS → *Transport Layer Security*

TLS creates a secure tunnel between browser and server



SSL vs TLS

Both are **protocols used to secure communication over the internet**

Encrypt data

Protect privacy

Prevent tampering

SSL (Old)

- Developed first
- Now **deprecated (not used anymore)**
- Less secure

TLS (Modern)

- Updated version of SSL
- More secure
- Used today in HTTPS

We say SSL, but in reality we are using TLS

How HTTPS Works



Browser connects to server

Server sends certificate

Browser verifies it

Secure connection established

Data is encrypted

They agree on a secret key before talking.



How Browser verifies it?

1. Server sends its certificate

When your browser connects to a website (like <https://example.com>), the server sends a **certificate**.

That certificate includes:

- Website domain (e.g., example.com)
- Server's public key
- Who issued it (Certificate Authority)
- A **digital signature**



2. Browser checks who issued it (CA)

Browsers already have a built-in list of trusted companies called **Certificate Authorities (CAs)** like:

- DigiCert
- Let's Encrypt
- GlobalSign

These are pre-installed in:

- Google Chrome
- Mozilla Firefox
- Safari

If the certificate is signed by one of these → browser says:
“Okay, I trust this issuer”



Digital Signature Check (IMPORTANT)

The CA signs the certificate using its private key

The browser uses the CA's public key (already stored inside the browser)

It checks the signature

If the signature matches → the certificate has not been changed

If it does not match → the browser shows **“Your connection is not private”**

Domain name check

Expiry check



What is a Certificate?

Digital identity of website

Issued by trusted authority

Contains:

- Domain name
- Public key

if you visit <https://google.com>

Browser checks the certificate

Public key is generally static (does NOT keep changing frequently)

Browser gets **public key**

Uses it to securely exchange data

Both agree on a **session key**

That session key is used for actual communication



TLS Session key

What the TLS session key is

- Created during HTTPS connection
- Shared **only between browser ↔ server**
- Used to:
 - Encrypt data
 - Decrypt data

It is invisible to your app code (you don't handle it in Node.js or Express.js)



What HTTPS Does NOT Do

Does NOT stop XSS

Does NOT fix bad code

Does NOT validate input

HTTPS protects data *in transit*, not your application logic.



Where HTTPS Helps

- Login forms
- Payment systems
- APIs
- Personal data

Nginx Role



Handles **HTTPS (TLS/SSL)**

Manages certificates

Acts as **reverse proxy**

Forwards request to Node app

Nginx acts as the secure gatekeeper.

Browser (Client)



Nginx (HTTPS, SSL/TLS)



Node/Express (Your App)



You don't "make" Nginx. You install and configure it.

Nginx is:

- A **separate software**
- Runs alongside your app
- Not part of Node/Express

`sudo apt install nginx`

You create a config in system paths like

`/etc/nginx/sites-available/myapp`

`/etc/nginx/sites-available/example`

Run it

`sudo systemctl start nginx`

Browser → Nginx (HTTPS) → Node/Express

Paths to the ssl keys - config file nginx - on server



```
server {  
    listen 443 ssl;  
    server_name example.com;  
  
    ssl_certificate /etc/ssl/certs/cert.pem;  
    ssl_certificate_key /etc/ssl/private/key.pem;  
}
```

```
# React frontend  
location / {  
    root /var/www/myapp/build;  
    index index.html;  
    try_files $uri /index.html;  
}  
  
# Node/Express API  
location /api {  
    proxy_pass http://localhost:4000;  
}
```




Where are actual keys

Stored as **separate files on the server** - These files live **outside your codebase**

`/etc/ssl/certs/cert.pem` → public certificate

`/etc/ssl/private/key.pem` → private key

Certificate file (public)

- Domain name
- Public key
- Issuer (CA)

Private key file (secret)

- Used to decrypt data
- Must NEVER be exposed



Redirect HTTP → HTTPS

Forces users to use secure connection

```
server {  
    listen 80;  
    server_name example.com;  
  
    return 301 https://$host$request_uri;  
}
```



Best Practices

Always use HTTPS

Redirect HTTP → HTTPS

Use trusted certificates

Keep TLS updated

References & Further Reading



SSL / TLS & HTTPS

- OWASP
<https://owasp.org>
→ Security best practices and HTTPS guidance
- Cloudflare Learning Center
<https://www.cloudflare.com/learning/ssl/what-is-ssl/>
→ Simple explanations of SSL/TLS

Web & HTTPS Concepts

- MDN Web Docs
<https://developer.mozilla.org>
→ HTTP, HTTPS, TLS explained clearly

Nginx & Configuration

- Nginx Official Docs
<https://nginx.org/en/docs/>
→ Configuration and HTTPS setup

Certificates & Tools

- Let's Encrypt
<https://letsencrypt.org>
→ Free SSL certificates
- Certbot
<https://certbot.eff.org>
→ Automatic HTTPS setup